



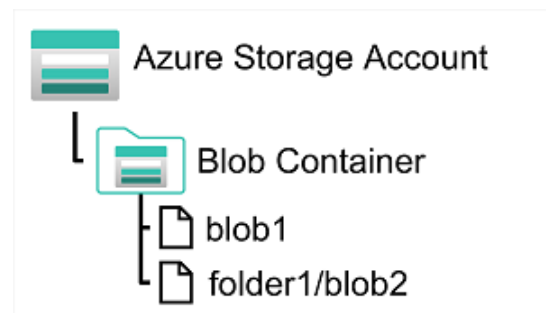
비관계형 데이터를 위한 Azure Storage

Azure Blob Storage

- 대량의 구조화되지 않은 데이터를 Blob 형태로 클라우드에 저장하는 서비스
- 애플리케이션은 Azure Blob Storage API를 통해 데이터를 읽고 쓰기 가능
- 액세스 계층을 자동으로 관리하는 기능 제공

Blob과 컨테이너

- 컨테이너
 - Blob은 **Storage Account** → **Container** → **Blob** 구조로 저장
 - 컨테이너는 관련 Blob을 그룹화하는 단위
 - 컨테이너 수준에서 액세스 제어 가능



- 가상 폴더 구조
 - Blob 이름에 "/" 문자를 사용해 가상 폴더처럼 계층화 가능
 - 실제 폴더가 아닌 이름 기반 네임스페이스
 - 따라서 폴더 단위 작업(권한 부여, 대량 삭제 등)은 불가능

Blob 유형

- 블록 Blob
 - 데이터 = 블록 단위(최대 4000MiB)
 - 최대 크기 = 190.7TiB
 - 자주 변경되지 않는 **큰 개체** 저장에 적합(대용량 파일)
- 페이지 Blob
 - 512바이트 페이지 단위
 - 임의 읽기/쓰기 최적화
 - 최대 8TB 저장 가능
 - 가상 머신 디스크 저장소로 사용

- 자주 업데이트되는 이진 데이터에 대한 고성능 액세스
- 자주 업데이트 및 임의로 액세스해야 하는 대규모 데이터 세트
- 추가 Blob
 - 추가 전용 (수정/삭제 불가)
 - 각 블록 최대 4MB
 - 최대 크기 $\approx$ 195GB
 - 로그 저장 등 순차적 추가에 적합

액세스 계층

- 핫(Hot) 계층
 - 기본값
 - 자주 액세스하는 데이터
 - 고성능, 비용 상대적으로 높음
- 쿨(Cool) 계층
 - 드물게 액세스
 - 성능은 낮고 비용 저렴
 - Blob을 핫 → 쿨, 쿨 → 핫으로 마이그레이션 가능
- 보관(Archive) 계층
 - 최저 비용, 가장 긴 지연 시간
 - 드물게 필요한 기록 데이터 저장
 - 읽기 위해 리하이드레이션(Rehydration) 필요 (몇 시간 소요 가능)

수명 주기 관리

- 정책 기반 자동 관리 가능
- Blob을 핫 → 쿨 → 보관 계층으로 자동 이동
- 일정 기간 후 자동 삭제도 가능

Azure Data Lake Storage Gen2

- **Azure Data Lake Store(Gen1)** → 분석 데이터 레이크용 계층적 스토리지 서비스
- **Gen2** = Azure Storage와 통합된 최신 버전
 - Blob Storage의 확장성 + 비용 효율성
 - Data Lake Store의 계층적 파일 시스템 & 분석 호환성

계층 구조 네임스페이스 지원

- Blob Storage 기반이지만 계층적 파일 시스템 제공
- 디렉터리/폴더 구조처럼 관리 가능
- 대규모 데이터 분석 및 처리에 최적화

분석 시스템과의 호환성

- **Azure Databricks** 같은 빅데이터/분석 시스템과 통합 가능
- 분산 파일 시스템으로 탑재하여 대량 데이터 처리 가능
- **Microsoft Fabric의 OneLake**도 Gen2 기반

생성 및 업그레이드

- 스토리지 계정에서 **계층 구조 네임스페이스 옵션을 켜야 Gen2 사용 가능**
- 새 스토리지 계정 생성 시 설정 가능
- 기존 스토리지 계정도 업그레이드 가능 (단방향 프로세스)
 - 업그레이드 후 다시 단일 네임스페이스로 되돌릴 수 없음

Fabric의 Microsoft OneLake 살펴보기

Microsoft OneLake 개요

- **Microsoft Fabric**에 포함된 단일 **통합 논리 데이터 레이크**
- **Azure Data Lake Storage (ADLS) Gen2** 기반
- 모든 Microsoft Fabric 테넌트에서 자동으로 제공됨
- 모든 분석 데이터의 중앙 리포지토리 역할 수행
- 구조적/비구조적 모든 형식의 파일 저장 가능
- 데이터 이동이나 중복 없이 여러 분석 엔진에서 동일 데이터 활용

조직 전체 데이터 레이크

- 과거: 여러 부서·그룹별로 별도의 데이터 레이크 운영
- 현재: OneLake를 통해 조직 전체가 하나의 데이터 레이크 공유

분산 소유권 및 협업

- 테넌트 내 작업 영역(workspace) 생성 가능
- 부서·팀별로 데이터 항목 관리 가능
- 거버넌스 경계는 유지하면서 협업 가능

개방형 및 호환성

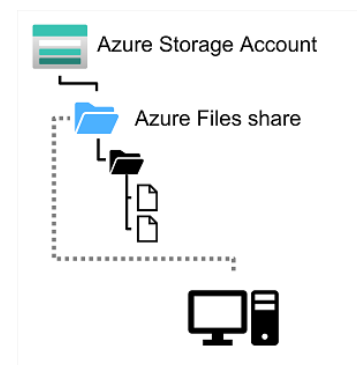
- ADLS Gen2 기반 + 델타 Parquet 형식 사용
- 기존 ADLS Gen2 API 및 SDK와 호환 → 현재 애플리케이션에서도 사용 가능

손쉬운 탐색

- OneLake 파일 탐색기를 통해 Windows 환경에서 간단히 탐색 가능

Azure Files

- 클라우드 기반 네트워크 파일 공유 서비스
- 온-프레미스의 전통적인 파일 공유(파일 서버)와 동일한 개념을 클라우드에서 제공
- 고가용성·확장성·하드웨어 관리 부담 감소 효과



스토리지 및 용량

- 스토리지 계정 내 Azure Files 생성 가능
- 단일 계정에서 최대 100TB 저장 가능
- 파일 크기: 최대 1TB
- 공유 파일당 최대 2,000개 동시 연결 지원
- 각 공유에 대해 할당량(Quota) 설정 가능

동기화 기능

- AzCopy 등 도구를 이용한 업로드 가능
- Azure 파일 동기화 서비스를 통해
 - 로컬 캐시된 파일과 Azure Files 간 자동 동기화 지원

성능 계층

- 표준 계층: HDD 기반, 일반적인 성능과 비용 절감 목적
- 프리미엄 계층: SSD 기반, 고성능 및 고처리량 제공 (비용 ↑)

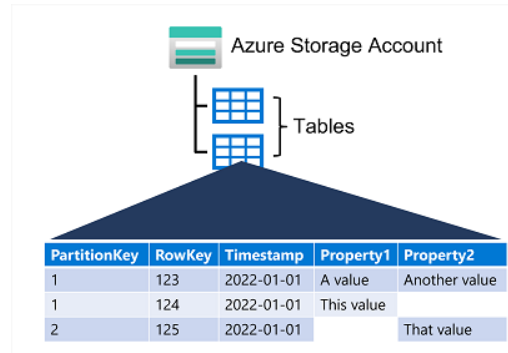
지원 프로토콜

- SMB (Server Message Block)
 - Windows, Linux, macOS 등 범용 지원
- NFS (Network File System)
 - 일부 Linux, macOS 지원

- 프리미엄 계층 필요
- 가상 네트워크 기반의 액세스 제어 필수

Azure 테이블 스토리지

- NoSQL 키-값 스토리지 서비스
- 테이블 = **행(Row) + 열(Column)** 구조이지만, 관계형 DB와는 다름
- 반정형 데이터 저장 가능 → 각 행마다 다른 열을 가질 수 있음
- 각 행은 반드시 고유 키(Partition Key + Row Key)를 가져야 함



데이터 구조

- **Partition Key + Row Key**
 - 파티션 키: 행이 속한 파티션 구분
 - 행 키: 파티션 내에서 행을 고유하게 식별
 - 두 키 조합 = 행의 유일한 식별자
- 행에는 자동으로 **Timestamp** 컬럼이 기록됨

관계형 DB와의 차이

- 외래 키, 관계, 조인, 뷰, 저장 프로시저 없음
- 비정규화된 구조 → 한 행에 엔터티의 모든 정보 포함
 - 예: 고객 테이블에 이름/전화번호/주소 여러 개를 **한 행에 다 포함**
 - (관계형 DB라면 여러 테이블에 분리 저장했을 것)

성능 및 확장성

- **파티션 기반 확장**
 - 동일한 파티션 키를 가진 데이터는 함께 저장 → 빠른 접근 가능
 - 파티션은 독립적으로 확장/축소 가능
- **쿼리 최적화**
 - 파티션 키를 조건에 포함하면 검색 범위를 줄여 성능 향상
 - 지원 쿼리:

- 포인트 쿼리 (PartitionKey + RowKey로 특정 행 검색)
- 범위 쿼리 (한 파티션 내 연속된 RowKey 블록 검색)